COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 1, 2026	**DATE SUBMITTED:**	Oct 1, 2026
NAME:	Formilleza, Huone Anthony	**INSTRUCTOR:**	Eng. Jimlord Quejado

ACTIVITY NO. _10.1_ **[Graphs]**

1. PROCEDURE OUTPUT

ILOA

Graph.h

```
1
2  #ifndef GRAPHA_H
3  #define GRAPHA_H
4
5  // stores adjacency list items
6  struct adjNode {
7      int val, cost;
8      adjNode* next;
9  };
10 // structure to store edges
11 struct graphEdge {
12     int start_ver, end_ver, weight;
13 };
14 class DiaGraph {
15     // insert new nodes into adjacency list from given graph
16     adjNode* getAdjListNode(int value, int weight, adjNode* head) {
17         adjNode* newNode = new adjNode;
18         newNode->val = value;
19         newNode->cost = weight;
20         newNode->next = head; // point new node to current head
21         return newNode;
22     } <- #18-25 adjNode* getAdjListNode(int value, int weight, adjNo
23     int N; // number of nodes in the graph
24
```

```
25  public:
26      adjNode** head; // adjacency list as array of pointers
27
28      // Constructor
29      DiaGraph(graphEdge edges[], int n, int N) {
30          // allocate new node
31          head = new adjNode*[N]();
32          this->N = N;
33          // initialize head pointer for all vertices
34          for (int i = 0; i < N; ++i)
35              head[i] = nullptr;
36          // construct directed graph by adding edges to it
37          for (int i = 0; i < n; i++) {
38              int start_ver = edges[i].start_ver;
39              int end_ver = edges[i].end_ver;
40              int weight = edges[i].weight;
41              // insert in the beginning
42              adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
43              // point head pointer to new node
44              head[start_ver] = newNode;
45          } <- #43-53 for (int i = 0; i < n; i++)
46      } <- #33-54 DiaGraph(graphEdge edges[], int n, int N)
47      // Destructor
48      // the sample used delete[] on a single node, so this walks each list and deletes node by node
49      ~DiaGraph() {
50          for (int i = 0; i < N; i++) {
51              adjNode* curr = head[i];
52              while (curr != nullptr) {
53                  adjNode* temp = curr;
54                  curr = curr->next;
55                  delete temp;
56              } <- #61-65 while (curr != nullptr)
57          } <- #59-66 for (int i = 0; i < N; i++)
58          delete[] head;
59      } <- #58-68 ~DiaGraph()
60  }; <- #16-69 class DiaGraph
61
62  #endif
```

Main.cpp

```cpp
#include <iostream>
#include "grapha.h"

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i);

int main() {
    // graph edges array
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}

void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
```

Output

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILOB1

Graph.h

```cpp
#ifndef GRAPHB1_H
#define GRAPHB1_H

#include <string>
#include <vector>
#include <iostream>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
}; <- #12-25 struct Edge

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G) {
    for (size_t i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    } <- #29-35 for (size_t i = 1; i < G.vertices(); i++)
    return os;
} <- #28-37 std::ostream& operator<<(std::ostream& os, const Gra
```

```cpp
template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {
    }
    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }
    // Return all edges in the graph
    auto& edges() const {
        return edge_list;
    }
    void add_edge(Edge<T>&& e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    } <- #56-63 void add_edge(Edge<T>&& e)
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        } <- #68-72 for (auto& e : edge_list)
        return edges_from_v;
    } <- #66-74 auto outgoing_edges(size_t v) const
    // Overloads the << operator so a graph can be written directly to a stream
    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
}; <- #40-83 class Graph

#endif
```

Main.cpp

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>
#include "graphb1.h"

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto& i : edges)
        for (auto& j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
} <- #9-27 auto create_reference_graph()

```

```cpp
template <typename T>
auto depth_first_search(const Graph<T>& G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            } <- #45-50 for (auto e : G.outgoing_edges(current_vertex))
        } <- #41-51 if (visited.find(current_vertex) == visited.end())
    } <- #36-52 while (!stack.empty())
    return visit_order;
} <- #30-54 auto depth_first_search(const Graph<T>& G, size_t dest)

template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
} <- #57-68 void test_DFS()

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
} <- #70-74 int main()
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\Desktop\Formilleza\ACTIVITY 10 Graphs\Graphs\iloB1>
```

ILOB2

Graph.h

```
1   #ifndef GRAPHB2_H
2   #define GRAPHB2_H
3   #include <string>
4   #include <vector>
5   #include <iostream>
6
7   template <typename T>
8   class Graph;
9   template <typename T>
10  struct Edge {
11      size_t src;
12      size_t dest;
13      T weight;
14      inline bool operator<(const Edge<T>& e) const {
15          return this->weight < e.weight;
16      }
17      inline bool operator>(const Edge<T>& e) const {
18          return this->weight > e.weight;
19      }
20  }; <- #12-23 struct Edge
21  template <typename T>
22  std::ostream& operator<<(std::ostream& os, const Graph<T>& G) {
23      for (size_t i = 1; i < G.vertices(); i++) {
24          os << i << ":\t";
25          auto edges = G.outgoing_edges(i);
26          for (auto& e : edges)
27              os << "{" << e.dest << ": " << e.weight << "}, ";
28          os << std::endl;
29      } <- #27-33 for (size_t i = 1; i < G.vertices(); i++)
30      return os; // the sample forgot this line
31  } <- #26-35 std::ostream& operator<<(std::ostream& os, const Gra
32
```

```cpp
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const {
        return V;
    }

    auto& edges() const {
        return edge_list;
    }

    void add_edge(Edge<T>&& e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    } <- #50-56 void add_edge(Edge<T>&& e)

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        } <- #60-64 for (auto& e : edge_list)
        return edges_from_v;
    } <- #58-66 auto outgoing_edges(size_t v) const

    // renamed to U so it does not clash with the class template parameter T
    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
}; <- #38-75 class Graph

#endif
```

Main.cpp

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <queue>
#include "graphb2.h"

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto& i : edges)
        for (auto& j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
} <- #9-27 auto create_reference_graph()

```

```cpp
template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        } <- #41-47 if (visited.find(current_vertex) == visited.end())
    } <- #36-48 while (!queue.empty())
    return visit_order;
} <- #30-50 auto breadth_first_search(const Graph<T>& G, size_t dest)
template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
} <- #53-64 void test_BFS()
int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
} <- #66-70 int main()
```

Output

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

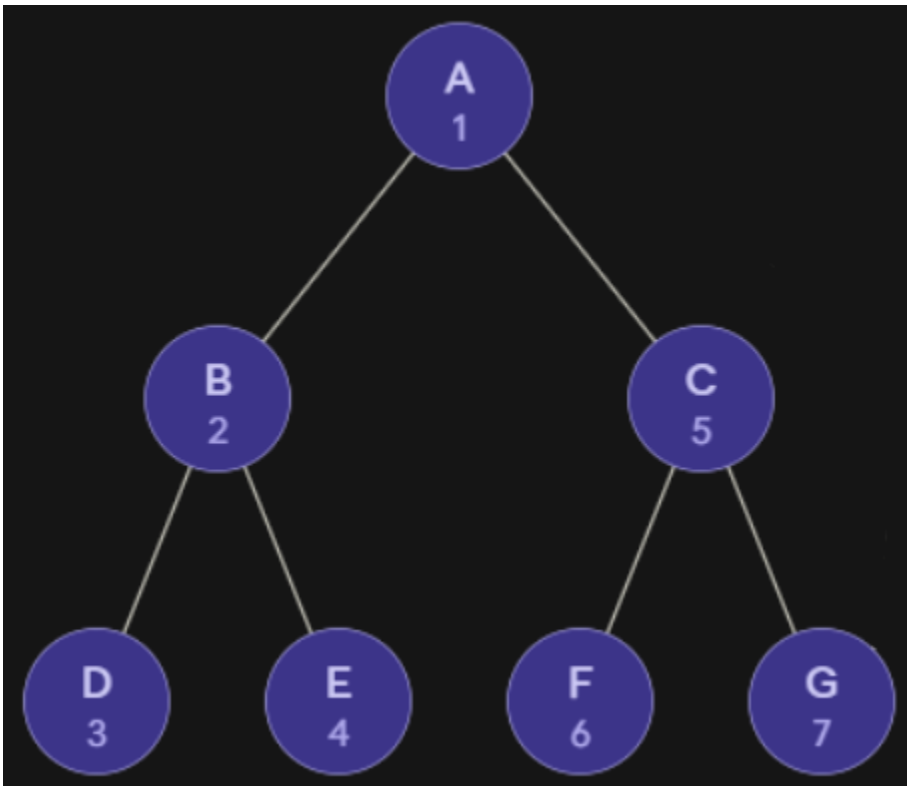
BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\Desktop\Formilleza\ACTIVITY 10 Graphs\Graphs\iloB2>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

the most helpful algorithm would be Depth First Search, or DFS. The user goes as deep as possible along a path from the starting location, backtracking to the closest location with an unexplored neighbor and repeating the process. This is effectively how DFS works, making it the best fit for the situation.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.



Visit order A B D E C F G

3. In the performed code, what data structure is used to implement the Breadth First Search?

the data structure used to implement Breadth First Search is a queue. A queue follows a first in first out principle, and processes the vertices in the order they were discovered, which creates the level by level discovery of BFS.

4. How many times can a node be visited in the BFS?

In BFS, a node can only be visited once. It may be placed in the queue multiple times, by different neighbors, but the visited set keeps track of this and ignores further instances of the node, resulting in it being processed only once.

Conclusion

I learned the basic principles of adjacency list and edge list, depth-first, and breadth-first search. DFS utilizes a stack structure and the algorithm traverses the graph deeper than wide. In contrast, BFS employs a queue structure as the primary mechanism and the graph traversal is shallow.

I ran the adjacency list, DFS, and BFS programs and have got the idea of the difference between stack- and queue-based graph algorithms. Using the same graph, the DFS algorithm returned 1 5 8 6 7 3 4 2 and BFS

produced 1 2 5 4 8 3 6 7, so it was interesting to see the difference in the output caused by the distinct data structures. Several logical mistakes had to be eliminated during the coding process.

The questions helped me to understand how to proceed with the code and the relation between the programs. DFS is used for traversing one path multiple times until the end is reached, and preorder is a DFS version for trees. In the case of BFS, the queue data structure is used for graph traversal when each vertex is visited only once.

I believe I performed well on the assignment because all the programs executed successfully and output the expected results. However, I have to practice more to become better at C++ and graph algorithms programming in particular. I have also learned how to proceed with writing code for graph algorithms without using ready examples from the course.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.